SDN Network Utilization Monitor

Subject Code: UE24CS252B

Name: Chandana Rajeswari P

SRN: PES2UG24AM043

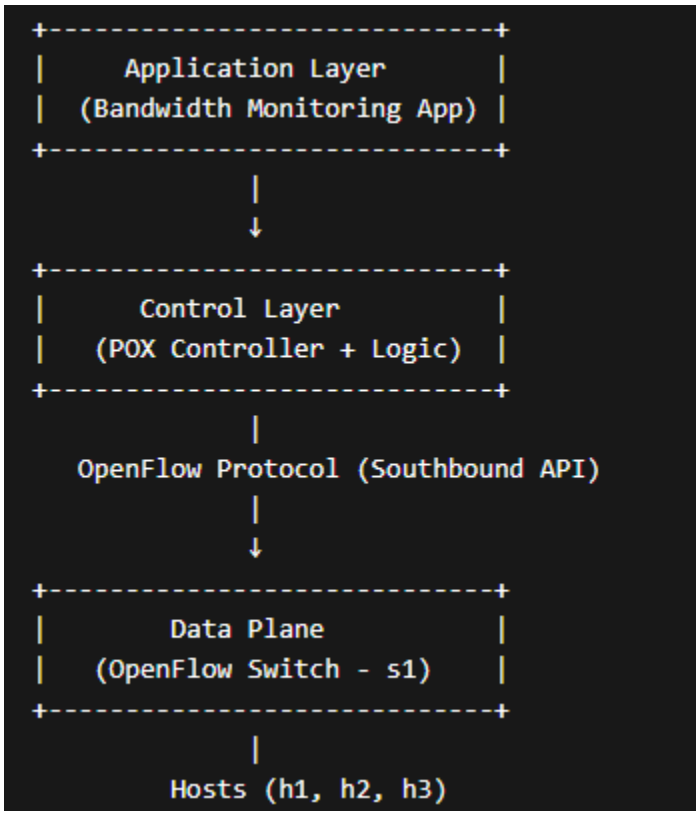
Table of Contents

Problem Statement

Measure and display bandwidth utilization across the network by collecting byte counters, estimating bandwidth usage, displaying utilization, and updating periodically.

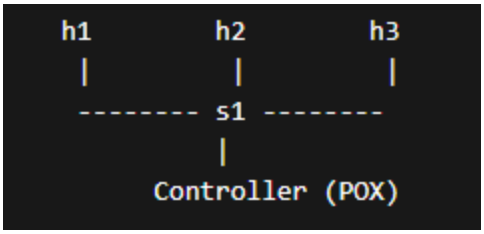
SDN Architecture

SDN separates control and data planes. Controller (POX) manages switches via OpenFlow.



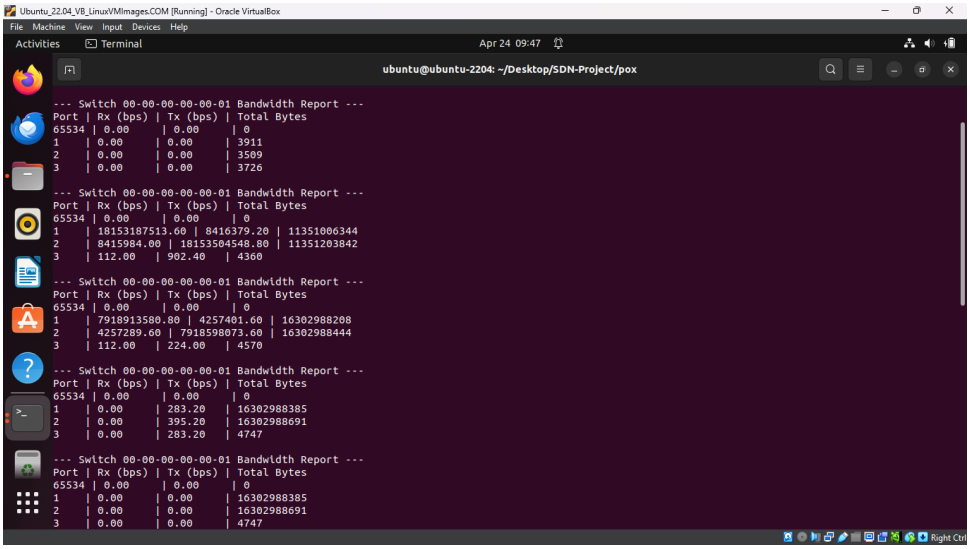
Mininet Topology Design

Topology: 1 switch (s1) connected to 3 hosts (h1, h2, h3). Used to test traffic and monitor utilization.



Controller Implementation

POX controller collects port statistics and calculates bandwidth using byte counters.

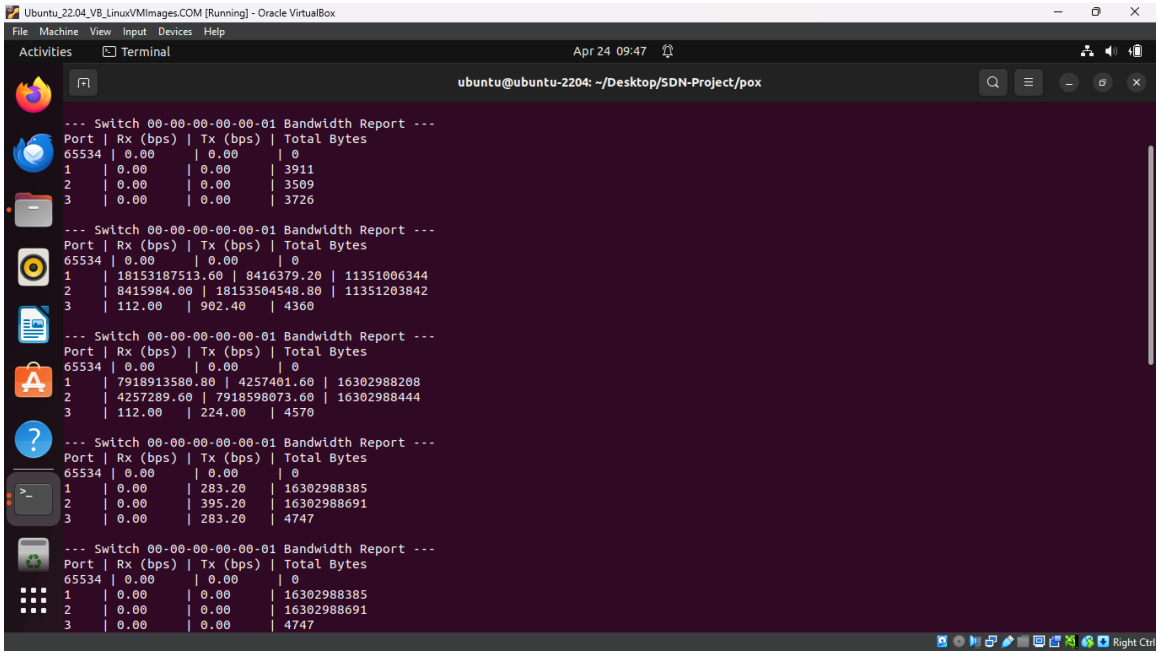


Flow Rule Management

Flow rules are installed dynamically to forward packets between hosts.

Functionality Implementation

The system is designed to monitor network utilization in real time using an SDN-based approach. When the Mininet topology is initialized, the hosts and switch establish connections with the POX controller.



Performance Evaluation

Tested using ping and iperf. Shows low usage initially and higher usage during traffic.

```
mininet> iperf h1 h2
*** Iperf: testing TCP bandwidth between h1 and h2
*** Results: ['30.8 Gbits/sec', '30.7 Gbits/sec']
```

```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=20.0 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.571 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.050 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.051 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.042 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.130 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.077 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.046 ms
^C
--- 10.0.0.2 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7133ms
rtt min/avg/max/mdev = 0.042/2.617/19.969/6.560 ms
```

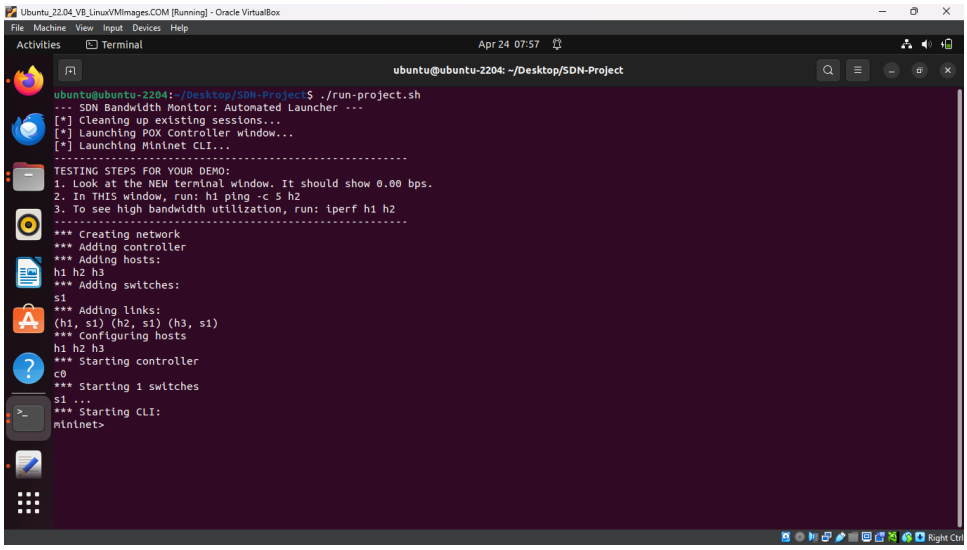
Python Script

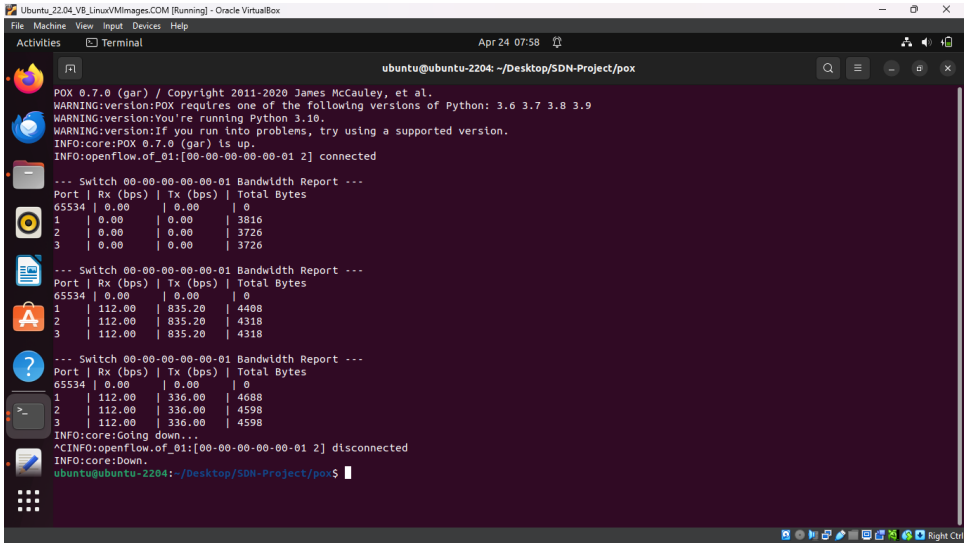
```
# Import core POX components

from pox.core import core

from pox.lib.util import dpidToStr

import pox.openflow.libopenflow_01 as of

from pox.lib.recoco import Timer

# Initialize the logger to display info in the POX console

log = core.getLogger()

class BandwidthMonitor(object):

  """

  This component implements an SDN-based network monitoring solution.

  It periodically queries OpenFlow switches for port statistics to

  calculate and display real-time bandwidth utilization[cite: 3, 39].

  """

  def __init__(self):

    # Register this class to listen for OpenFlow events (e.g., connection, stats)
```

```
        core.openflow.addListeners(self)
            # Dictionary to store previous byte counters: {dpid: {port_no: (rx_bytes, tx_bytes)}}
        # This is essential for calculating the difference (delta) over time[cite: 36, 39].
        self.stats = {}
            # Create a recurring timer to trigger the statistics request every 5 seconds[cite: 10, 39].
        # Functional Requirement: Update statistics periodically[cite: 11].
        Timer(5, self._request_stats, recurring=True)
    def _request_stats(self):
        """
        Iterates through all switches connected to the controller and sends
        an OpenFlow Port Statistics Request[cite: 4, 10].
        """
        for connection in core.openflow.connections:
            # Construct and send the ofp_stats_request message
            # body=of.ofp_port_stats_request() targets the switch port counters
            connection.send(of.ofp_stats_request(body=of.ofp_port_stats_request()))
    def _handle_PortStatsReceived(self, event):
        """
        Event handler triggered when a switch responds with its port statistics[cite: 10].
        Calculates throughput and displays the utilization report[cite: 34, 39].
        """
        dpid = event.connection.dpid
        # Initialize stats storage for a new switch connection
        if dpid not in self.stats:
            self.stats[dpid] = {}
```

```
        # Display header for the current switch report [cite: 30, 36]
        print("\n--- Switch %s Bandwidth Report ---" % dpidToStr(dpid))
        print("Port | Rx (bps) | Tx (bps) | Total Bytes")
            for f in event.stats:
          # Retrieve previous statistics for this specific port; default to current if first time
          prev_rx, prev_tx = self.stats[dpid].get(f.port_no, (f.rx_bytes, f.tx_bytes))
             # Logic to calculate bandwidth utilization:
          # Formula: ((Current_Bytes - Previous_Bytes) * 8 bits) / 5 seconds
          rx_bps = (f.rx_bytes - prev_rx) * 8 / 5
          tx_bps = (f.tx_bytes - prev_tx) * 8 / 5
          # Update the history with current byte counts for the next interval
          self.stats[dpid][f.port_no] = (f.rx_bytes, f.tx_bytes)
          # Display result (Functional Correctness - Performance Observation) [cite: 25, 39]
          print(f"{f.port_no:<4} | {rx_bps:<8.2f} | {tx_bps:<8.2f} | {f.rx_bytes + f.tx_bytes}")
def launch():
    """
    POX launch function to register and start the component.
    """
    core.registerNew(BandwidthMonitor)
```

Screenshot of Demo





GitHub Repository Link

https://github.com/ChandanaRajeswariP/SDN-Network-utilization-monitor

Conclusion

This project demonstrates the use of SDN for real-time network utilization monitoring using Mininet and a POX controller. The system collects port statistics, calculates bandwidth usage, and updates it periodically. The results show that SDN provides efficient and centralized control for monitoring network performance.